🐍 Python for Data Science — visual cheat sheet

Theory · Python · NumPy · pandas · Traps · → = result · 💥 = error

THEORY (paper part)

1 Software lifecycle = 6 phases

Requirements → Design → Implementation → Testing → Deployment → Maintenance

What should it do?	How built?	Write code	Works?	Release	Fix / add

🧠 **"Rami Drinks Iced Tea During Matches" → R D I T D M**

Software = **instructions** telling the computer what to do. **Application** (Instagram, Word, games) vs **System** (Windows, Android).

2 Information System = 5 components

🖥 Hardware	💾 Software	📊 Data	👥 People	🔁 Process
servers	website	grades	students, staff	how grades are entered

🧠 **"Happy Students Do Python Projects" → H S D P P**

IS ≠ just software. Food delivery app → *Data* = order, address, menu prices.

3 Data science + why Python

Collect → Store → Analyse 🐍 → Decide

🧠 **"Cats Sleep After Dinner" — Python = step 3**

Python = **interpreted**: the interpreter reads **line by line, top → bottom**, no compile step → notebooks are instant. Readable, versatile (data, web, automation, AI, games), huge community. Java: 4 lines to print · Python: `print("Hello!")`.

PYTHON BASICS

4 4 types + conversions

int	float	str	bool
20, -5	19.9, 3.0	"Amine"	True False

```
type(7)        → <class 'int'>
int("7")       → 7      int(3.9)  → 3 (cuts)
float("2.5")   → 2.5    str(20)   → "20"
int("3.5")     💥 ValueError → int(float("3.5"))
bool(0) → False  bool("") → False  7 == "7" → False
```

5 input() & print()

🧠 **input() ALWAYS gives a str 🧵 → convert or cry**

```
a = input("Age: ")     # types 20 → "20" (str)
"5" + "3"  → "53"     int("5") + int("3")  → 8
"4" * 2    → "44"     "5" + 3  💥 TypeError
print("A", "B", sep="-")      → A-B
print(1, end=" "); print(2)   → 1 2
print(f"{name} has {x:.2f} TND")   # 2 decimals
f"{1234567:,}" → 1,234,567   f"{0.256:.1%}" → 25.6%
```

6 Operators

7 / 2	7 // 2	7 % 2	2 ** 3	6 / 2
3.5	3	1	8	3.0

🧠 **/ → always float · // → floor · % → leftovers**

```
n % 2 == 0     # even?       17 // 5 → 3   17 % 5 → 2
i // ncols → row      i % ncols → column   ⭐ (card 21)
2 + 3 * 2 ** 2 → 14   # ** then * then +
==  compare      =  assign
and (both) · or (one) · not (flip) · 1 < x < 18 ok
```

7 if / elif / else

```
if score >= 16:
    g = "Excellent"
elif score >= 12:        # first True wins ✋ rest skipped
    g = "Good"
else:
    g = "Fail"
x = "adult" if age >= 18 else "minor"   # 1 line
```

🧠 **Colon : + 4 spaces. Test the SPECIAL case first (n == 0 before n % 2 == 0)**

8 range & loops

range(5) → 0 1 2 3 4 range(2,10,3) → 2 5 8 ~~10~~

🧠 **range stops BEFORE the door 🚪 (stop excluded)**

```
total = 0
for i in range(1, 5):
    total += i            # running total → 10
count = 0
for s in scores:
    if s >= 10: count += 1     # counter
for i, f in enumerate(fruits): ...   # index + item
while n <= 3: n += 1       # forget += → infinite
continue skip this turn · break leave loop
```

9 Lists — index ruler

+	0	1	2	3	4
lst	10	20	30	40	50
–	-5	-4	-3	-2	-1

```
lst[0] → 10  lst[-1] → 50  lst[1:4] → [20, 30, 40]
lst[:2] → [10, 20]  lst[::2] → [10, 30, 50]
lst[::-1] reversed
append(x) insert(i,x) extend([..]) remove(x) pop()
sort() in place · sorted(lst) new list · len sum
min max
"x" in lst · lst.count(x) · lst.index(x) · lst[9]
💥 IndexError
```

10 list vs tuple vs dict vs set

	list	tuple	dict	set
write	`[1,2]`	`(1,2)`	`{"k":1}`	`{1,2}`
change?	✅	❌ locked	✅	✅
access	lst[0]	t[0]	d["k"]	—
use for	changing data	fixed (x, y), RGB, shape	key → value	no duplicates

```
x, y = (3, 7)        # unpacking   t = (5,) 1-item
tuple
```

11 Dictionary

```
d = {"Tunis": 3, "Sfax": 1}
d["Sousse"] = 2          # add new key (goes at the
end)
d["Tunis"] += 1          # update → 4
for k, v in d.items():   print(k, v)
"Tunis" in d → True (tests KEYS)   d.get("X", 0) →
0
d["X"]  💥 KeyError    del d["Sfax"]
d.pop("Sfax")
d.keys()  d.values()  len(d)
# counting pattern ⭐
counts[w] = counts.get(w, 0) + 1
```

12 Nested list (board) + strings

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]

```
board[row][col] = "X"      for row in board:
print(" | ".join(row))
" Hi ".strip() → "Hi"  "tunis".title() → "Tunis"
.upper() .lower()
"a,b".split(",") → ['a','b']   "-".join(["a","b"])
→ "a-b"
.replace("l","L")  .startswith("a")  "42".isdigit()
"ell" in "hello"
```

🧠 strings are immutable → s = s.upper() (reassign!)

13 Functions, lambda, comprehension

```
def stock_value(stock, prices, rate=0):   # default
    total = 0
    for name, qty in stock.items():
        total += qty * prices[name]
    return round(total, 2)       # return STOPS +
gives back
sq = lambda x: x ** 2           # sq(4) → 16
[x**2 for x in range(5)]          → [0, 1, 4, 9,
16]
sorted([n for n, q in stock.items() if q < 5])
```

🧠 return gives a value · print only shows · no return → None

14 import

```
import random
random.randint(1, 100)   # 1 AND 100 included
random.choice(lst)   random.shuffle(lst)
import math
math.sqrt(16) → 4.0  math.floor(3.7) → 3
math.ceil(3.2) → 4
from math import pi       # use pi directly
import numpy as np  ·  import pandas as pd  ·
import matplotlib.pyplot as plt
```

Library = tested ready-made code → saves time, reliable.

NUMPY

15 list vs array

Python list	NumPy array
mixed types `["a", 5, True]`	ONE type only
slow, more memory	fast, less memory
`[1,2]*2 → [1,2,1,2]`	`arr*2 → [2 4]` ⚡ vectorised

1D = vector **2D = matrix** **3D+ = tensor**

16 Create arrays

```
np.array([3, 2, 5])            # from list
np.array([[1, 2], [3, 4]])     # 2D from list of
lists
np.zeros((2, 3))   # shape = TUPLE (( ))
np.ones((2, 2))   np.full((2, 2), 7)
np.arange(4)        → [0 1 2 3]
np.arange(-3, 4)    → [-3 -2 -1 0 1 2 3]
np.arange(-3, 4, 3) → [-3 0 3]
np.random.random((2, 4))   # floats in [0, 1)
np.linspace(0, 1, 5) → [0. 0.25 0.5 0.75 1.]
```

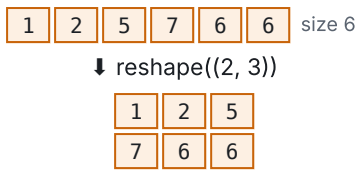
17 Attributes = NOUNS (no parentheses!)

🧠 NOUNS have no () : shape size ndim dtype · VERBS have () : sum() mean() reshape()

```
visits = np.array([...28 values...])
visits.shape → (28,)   visits.size → 28
visits.ndim  → 1       visits.dtype → int64
weeks = visits.reshape((4, 7))
weeks.shape  → (4, 7)  = (rows, cols)   weeks.ndim
→ 2
np.zeros((3, 5)).shape → (3, 5)   .size → 15
```

shape	size	ndim	dtype
dimensions (tuple)	nb of elements	nb of axes	element type

18 reshape / flatten



🧠 rows × cols MUST = size → 2×3 ✅ 3×2 ✅ 3×3 💥

```
a.reshape((3, 3))  💥 ValueError: cannot reshape array of size 6 into shape (3,3)
a.flatten()   # back to 1D     a.reshape(2, -1)  # -1 = "you compute"
```

19 AXIS ⭐ (the one everybody forgets)

	col 0	col 1	col 2	axis=1 ➡
row 0	1	2	3	**6**
row 1	4	5	6	**15**
axis=0 ⬇	**5**	**7**	**9**	sum()=21

```
a.sum(axis=0) → [5 7 9]  # ⬇ down → 1 result PER COLUMN
a.sum(axis=1) → [ 6 15] # ➡ across → 1 result PER ROW
```

🧠 axis=0 ⬇ "zero falls down" · axis=1 ➡ "one line at a time"

weeks (4 weeks × 7 days): `weeks.mean(axis=1)` = per **week** (4 values) · `weeks.mean(axis=0)` = per **weekday** (7 values). Same for `.mean() .max() .min()`.

20 Index & slice arrays

```
sales = np.array([120, 150, 95, 180, 160, 210, 175])
sales[0] → 120  sales[-1] → 175  sales[1:5] → [150 95 180 160]
a = np.array([[1, 2, 3], [4, 5, 6]])
a[1, 2] → 6  (row 1, col 2)   a[0] → [1 2 3] (row)
a[:, 0] → [1 4] (column 0)     a * 10  a + a  ⚡ element-wise
```

21 Stats + argmax trick ⭐

```
sales.sum() → 1090  sales.mean() → 155.71…  .min() .max() .std()
np.median(sales) → 160.0   np.sort(sales) .round(1)
i = sales.argmax() → 5   # the POSITION, not the value!
days[sales.argmax()] → 'Saturday'
# 1D index → position in a (4, 7) grid
i = visits.argmax()          # e.g. 26
week = i // 7 + 1  → 4      # row (+1 for humans)
day  = days[i % 7] → 'Sat'  # column
```

🧠 argMAX = WHERE is the max · max = WHAT is the max

22 Boolean masks

```
mask = sales > 150
→ [False False False True True True True]
sales[mask] → [180 160 210 175]   # the values
days[mask]  → ['Thu' 'Fri' 'Sat' 'Sun']  # their names
mask.sum()  → 4      # True = 1 → COUNT
days[sales >= sales.mean()]
sales[(sales > 100) & (sales < 170)]   # & | ~ + ( )
a[a > 2] → [3 4 5 6]   # 2D mask → 1D result
np.where(sales > 150, "good", "low")
```

23 dtype & type coercion ladder ⭐

bool → **int** → **float** → **str** 👑

🧠 ONE type per array → the highest type present wins · str eats everything

```
np.array([True, False, 42]).dtype   → int64
np.array([0, 42, 42.42]).dtype      → float64
np.array([2, 4.0, True]).dtype      → float64
np.array([1, "2", 3.0]).dtype       → <U32  (text)
np.array([True, "Boop", 42]) → ['True' 'Boop' '42']
np.array(["Introduction","to","NumPy"]).dtype → <U12 (12 = longest word)
np.array([1.3, 5.7], dtype=np.float32)    # choose
b.astype(np.int32)    # convert: True→1 False→0
```

int64	int32	float64	<U12
default int	32 bits → 2^{32} = 4,294,967,296 values	default decimal	string ≤ 12 chars

PANDAS

24 DataFrame anatomy

index ⬇	order_id	city	quantity
0	0070	Tunis	1
1	0059	Sfax	3
2	0086	Sousse	2

```
df["city"]            # 1 column = a Series (green)
df.loc[1, "city"] → 'Sfax'   # 1 cell by label
pd.DataFrame({"city": ["Tunis", "Sfax"], "q": [1,
3]})
```

🧠 **DataFrame = table · Series = one column · index = row labels · 1 dtype per column**

25 Load

```
import pandas as pd
raw = pd.read_csv("data/orders.csv")
df = raw.copy()                      # ⭐ never
touch raw
pd.read_csv("f.csv", sep=";")
pd.read_csv("f.csv", parse_dates=["order_date"])
pd.read_excel("f.xlsx", sheet_name="Sales")
# Colab: from google.colab import files;
files.upload() → "/content/f.csv"
```

read_csv: splits columns, guesses dtypes, `""` / `"N/A"` → **NaN**.

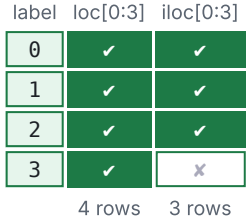
26 Explore — 1 question = 1 command

Question	Command
How big?	`df.shape` → (12060, 10) **no ()**
How many cells?	`df.size` → 120600 (= rows × cols)
How many rows?	`len(df)` → 12060
First / last rows?	`df.head(10)` · `df.tail()`
Types + missing?	`df.info()` ⭐
Weird numbers?	`df.describe()` (max = 999 ⚠)
Column names / types?	`df.columns` · `df.dtypes`
Categories & counts?	`df["c"].value_counts()`
Distinct values?	`df["c"].unique()` · `.nunique()` · `df.nunique()`
Frequencies in order?	`df["q"].value_counts().sort_index()`

🧠 **NOUNS no (): shape size columns dtypes · VERBS (): head() info() describe()**

27 Select: [ ] · loc · iloc

```
df["city"]                  # Series
df[["order_id", "city"]]    # DataFrame ⭐ DOUBLE
brackets
df.drop(columns="status")  # all except
```



🧠 **loc = Label, end incLuded · iloc = integer, end excluded (like Python)**

```
df.loc[0, "city"]   df.loc[mask, ["id", "q"]]
df.iloc[0:5, 0:3]   df.iloc[-1]
```

28 Filter rows (masks)

```
mask = (df["status"] == "Delivered") & (df["city"]
== "Tunis") & (df["q"] >= 3)
df[mask]   ·   df.loc[mask, ["order_id", "q"]]
df.query("status == 'Delivered' and q >= 3")   #
same rows ✅
df[df["city"].isin(["Tunis", "Sfax"])]     # OR on
values
df[df["q"].between(2, 5)]                  # 2..5
both included
df[~df["q"].between(1, 50)]                # ~ =
NOT
df[df["payment"].isna()]                   #
missing rows
mask.sum()  or  len(df[mask])              # count
```

🧠 **pandas speaks SYMBOLS: & | ~ — never and/or/not · ( ) around EACH condition**

29 Sort & top

```
df.sort_values("price", ascending=False)
df.sort_values(["region", "q"], ascending=[True, False])
df.nlargest(5, "price")   df.nsmallest(5, "price")
df["price"].idxmax()  → label of the max → df.loc[...]
s.idxmax()  # best group after groupby
df.reset_index(drop=True)   # 0..n-1 again
```

30 Missing values — decision tree ⭐

category?	→ label `.fillna("Unknown")` or mode `.fillna(s.mode()[0])`
number?	→ median `.fillna(s.median())` / median of the group (card 36)
key field?	→ drop rows `df.dropna(subset=["order_date"])`
column mostly empty?	→ ask business, maybe `df.drop(columns=...)`
0 everywhere?	❌ NEVER → fake values, biased averages

```
df.isna().sum()                          # per column
(df.isna().mean() *
100).sort_values(ascending=False)   # % ⭐
df.isna().sum().sum()                    # total
df[df.isna().any(axis=1)]                # rows with ≥ 1 NaN
df["c"] = df["c"].fillna(...)            # ⭐ assign back
```

🧠 median > mean when outliers (robust) · document EVERY choice

31 NaN facts

```
NaN is a FLOAT → int column with NaN becomes 1.0, 2.0 (float64)
pd.DataFrame({"q": [2, None, 5]})["q"].dtype → float64
mean() / sum() SKIP NaN →  [2, NaN, 5, 5].mean() → 4.0
isna() == isnull()   (same)   ·   notna() = opposite
value_counts() ignores NaN → value_counts(dropna=False)
astype("int64") with NaN  💥 IntCastingNaNError → fill/drop first
```

32 Duplicates

```
df.duplicated().sum()              # identical rows → e.g. 60
df[df.duplicated(keep=False)]      # see ALL copies
df = df.drop_duplicates()          # keeps the 1st
df["order_id"].duplicated().sum()
df["order_id"].is_unique
```

duplicated ROW	duplicated ID
same in every column → delete ✅	same key, different content → investigate 🔍

Not removed → revenue counted twice.

33 Types & dates

```
df["d"] = pd.to_datetime(df["d"], errors="coerce")
# bad → NaT
df = df.dropna(subset=["d"])
df["p"] = pd.to_numeric(df["p"], errors="coerce")
# bad → NaN
df["q"] = df["q"].astype("int64")       # AFTER NaN gone
df["city"] = df["city"].astype("category")
d = df["d"].dt
d.year  d.month  d.day  d.quarter  d.month_name()
d.day_name()
d.dayofweek >= 5   # weekend (Mon=0 … Sun=6)
df[(df["d"] >= "2025-01-01") & (df["d"] <= "2025-03-31")]
df[df["d"].dt.month.isin([12, 1, 2])]   # winter
```

🧠 coerce = "can't read it? make it missing" (NaT / NaN) instead of 💥

34 Text cleaning

`" tunis "` `"TUNIS"` `"Tunis "` → `"Tunis"`

```
df["city"].nunique()                                  # 50 spellings
df["city"] = df["city"].str.strip().str.title()
df["city"].nunique()                                  # 10 ✅
.str.lower()  .str.upper()  .str.contains("Ben")
.str.replace("a", "b")
df["city"].replace({"Gabes": "Gabès"})
```

🧠 .str. = apply a string method to the WHOLE column

35 Outliers & business rules

```
# rule: 1..50 units per line → remove impossible rows
df = df[df["q"].between(1, 50)].copy()
# rule: age 18..100 → error = missing, keep the row!
bad = (df["age"] < 18) | (df["age"] > 100)
df.loc[bad, "age"] = np.nan
df["age"] = df["age"].fillna(df["age"].median())
# unknown key (not in catalogue) → remove
df = df[df["med_id"].isin(meds["med_id"])]
# statistical (IQR): outside [Q1 − 1.5·IQR , Q3 + 1.5·IQR]
```

🧠 business rule beats statistics · "999: real or typo? who confirms?"

36 transform vs agg ⭐

.groupby("p")["price"].median()	.groupby("p")["price"].transform("median")
P1 10 P2 40 1 value per group (short)	P1 10 P2 40 P1 10 P2 40 same length as df, aligned ✅

```
med = df.groupby("product_id")["unit_price"].transform("median")
df["unit_price"] = df["unit_price"].fillna(med)
```

🧠 transform = "give every row its group's value" → fillna

37 Cleaning workflow + export

Raw (keep) → **Explore** → **Decide** → **Clean** → **Validate** → **Export**

🧠 **"Real Experts Decide, Clean, Validate, Export"**

```
assert df.isna().sum().sum() == 0     # stops if
False
df.to_csv("clean.csv", index=False)   # no
"Unnamed: 0" ⭐
df.head(500).to_excel("sample.xlsx", index=False)
check = pd.read_csv("clean.csv", parse_dates=["d"])
check.shape == df.shape → True
print(f"{(len(raw)-len(df))/len(raw)*100:.1f} %
removed")
```

Lab 2: 12,060 → −60 dup → −20 dates → −15 qty 999 → **11,965** (0.8 %).

38 New columns & conditions

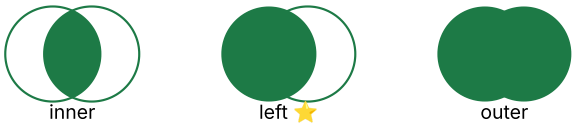
```
df["revenue"] = df["q"] * df["price"]          # ⚡
every row
df["profit"] = df["revenue"] - df["q"] * df["cost"]
df["sign"] = np.where(df["profit"] >= 0, "+", "-")
# 2 classes
conds = [df["profit"] > 500, df["profit"] >= 100,
df["profit"] >= 0]
df["level"] = np.select(conds, ["High", "Med",
"Low"], default="Loss")  # like elif
df["age_group"] = pd.cut(df["age"], bins=[17, 29,
49, 64, 100],
                         labels=["18-29", "30-49",
"50-64", "65+"])     # (a, b]
df["pay"] = df["pay"].map({"Cash": "Cash at
counter"})   # dict → missing key = NaN
df["tax"] = df["rev"].apply(lambda x: x * 0.05)
# each value
df.apply(func, axis=1)
# each ROW
```

39 groupby = split → apply → combine

split by city → **sum each group** → **1 row per city**

```
df.groupby("city")
["revenue"].sum().sort_values(ascending=False)
df.groupby("city")["revenue"].agg(["sum", "mean",
"count"])
df.groupby("category").agg(                  # named
agg ⭐
    sales=("sale_id", "count"),
    revenue=("revenue", "sum"),
    profit=("profit", "sum"))
s = df.groupby("city")["revenue"].sum()
s / s.sum() * 100          # share %
s.idxmax()                 # best group
s.rank(ascending=False)
df.groupby(["region", "cat"])
["profit"].sum().reset_index()
(df["presc"] == "Yes").mean() * 100        # % of
rows that are Yes
# category columns → groupby(..., observed=True)
```

40 merge (join on a key)




```
full = sales.merge(products, on="Product_ID",
how="left")   # keep ALL sales
# left_on="pid", right_on="Product_ID" if names
differ
sales["Product_ID"].isin(products["Product_ID"]).al
l()     # keys OK?
price =
sales["pid"].map(products.set_index("Product_ID")
["List_Price"])
pd.concat([jan, feb])     # stack tables
```

41 pivot, stats, plots

```
pd.pivot_table(df, values="rev", index="cat",
columns="Quarter", aggfunc="sum")
pd.crosstab(df["region"], df["pay"],
normalize="index")   # % per row
df["p"].agg(["mean", "median", "min", "max", "std",
"var"])
df["p"].quantile([0.25, 0.5, 0.75])      # Q1
median Q3
df[["q", "price", "profit"]].corr()     # −1 … +1
s.plot(kind="bar"); plt.show()   # line · hist ·
pie · barh
```

🧠 **mean > median → few huge values (right skew) · correlation ≠ causation**

TRAPS & ERRORS

42 ❌ wrong → ✅ right

❌ wrong	✅ right
`df.shape()`	`df.shape`
`df["a", "b"]`	`df[["a", "b"]]`
`df.dropna() (alone)`	`df = df.dropna()`
`df[a == 1 and b > 2]`	`df[(a == 1) & (b > 2)]`
`np.zeros(2, 3)`	`np.zeros((2, 3))`
`"5" + 3`	`int("5") + 3`
`int("3.5")`	`int(float("3.5"))`
`astype(int) with NaN`	`fillna / dropna first`
`fillna(0) everywhere`	`label / median / drop`
`to_csv("f.csv")`	`to_csv("f.csv", index=False)`
`if n % 2 == 0 before n == 0`	`test n == 0 first`
`if x = 5:`	`if x == 5:`
`b = a (same list!)`	`b = a.copy()`
`range(1, 10) = 1..10`	`1..9 (stop excluded)`
`reshape to wrong size`	`rows × cols = size`
`max() for the position`	`argmax()`

43 Error names

SyntaxError	missing `:`, bracket, quote
IndentationError	block not indented
NameError	variable undefined / typo / cell not run
TypeError	"5" + 3 · modify a tuple · len(5)
ValueError	int("abc") · bad reshape · and between Series
IndexError	lst[10] on a short list
KeyError	dict key / column name not found (case, spaces!)
ZeroDivisionError	x / 0
FileNotFoundError	wrong path (Colab: /content/...)

44 Practical exam order ⭐

Load+look → **% missing** → **Duplicates** → **Text** → **Dates** → **Rules** → **Fill** → **assert** → **Analyse** → **Export**

🧠 **"Lazy Monkeys Drink Tea During Rainy Fridays And Always Eat"**

✅ print before → after counts · ✅ one-line `# why` per decision · ✅ Markdown answers in sentences · ✅ Restart & Run all before submitting.

Mock DS → which card

Mock DS question	Card
Theory Q1–Q2 lifecycle, IS	1, 2
Q3 input type · Q4 // % · Q5 immutable	5, 6, 10
Q6 dtype [2, 4.0, True] · Q7 reshape(3,5)	23, 18
Q8 info() · Q9 shape vs size	26
Q10 masks · Q11 coerce · Q12 loc/iloc	28, 33, 27
P1 "4"*2, / · P2 range step, end=" "	5, 6, 8
P3 slicing · P4 dict	9, 11
P5 shape, size, axis, a[1,2], a[a>2]	17, 19, 20, 22
P6 None → float, duplicated, strip/title	31, 32, 34
S1–S4 float/NaN, missing, loc/iloc, transform	31, 30, 27, 36
C1 grade() + counter + average	7, 8, 13
C2 cleaning pipeline on orders	32, 34, 35, 36, 38, 39, 37
PC Ex1 dict loop, function, sell(), sorted list	11, 13, 7
PC 2.1 shape size ndim dtype reshape	17, 18
PC 2.2 mean axis=0 / axis=1	19
PC 2.3 argmax → week // 7, day % 7	21
PC 2.4 mask count, coercion	22, 23
PC 3.1 explore	25, 26
PC 3.2 clean (10 steps)	30–37
PC 3.3 merge, groupby, share, nlargest, dt.month, query, pd.cut	40, 39, 29, 33, 28, 38
PC 3.4 export + reload	37